Unit III

Graphs

1

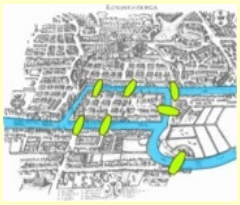
History

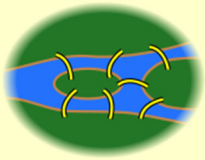
- The history of graphs dates back to 1736 in what is now referred to as the classical *Koenigsberg bridge problem*.
- Euler solved the problem with a theory that laid the foundation for graph theory.

2

Koenigsberg bridge

- The city of Königsberg in Prussia (now Kaliningrad, Russia) was set on both sides of the Pregel River, and included two large islands which were connected to each other and the main land by seven bridges.

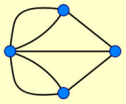




3

Koenigsberg bridge problem

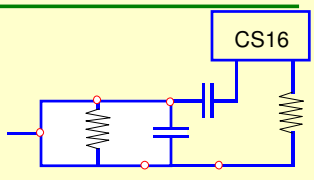
- The problem was to find a walk through the city that would cross each bridge once and only once. The islands could not be reached by any route other than the bridges, and every bridge must have been crossed completely every time; one could not walk halfway onto the bridge and then turn around and later cross the other half from the other side. The walk need to start and end at the same spot.
- Euler proved that the problem has **no** solution.

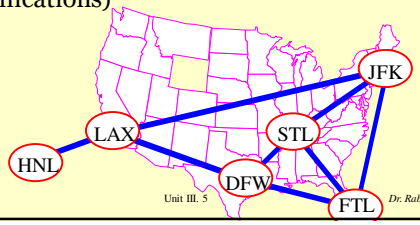


4

Applications

- Electronic circuits
- Networks (roads, flights, communications)





5

Some Problems that can be Represented by a Graph

- Computer networks
- Airline flights
- Road map
- Course prerequisite structure
- Tasks for completing a job
- Flow of control through a program
- Solving research problems
- Many more

6

Graphs

- $G = (V, E)$
- V is the vertex set.
- Vertices are also called *nodes* and *points*.
- E is the edge set.
- Each edge connects two different vertices.
- Edges are also called *arcs* and *lines*.
- Directed edge has an orientation (u, v).

$u \longrightarrow v$

7

Graphs

- Undirected edge has no orientation (u, v).

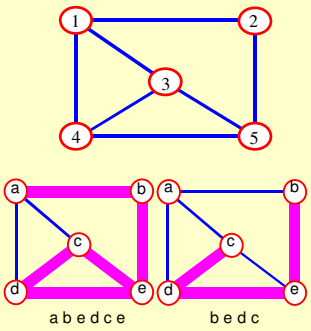
u —— v

- Undirected graph => no oriented edge.
- Directed graph => every edge has an orientation.

8

Terminology: Path

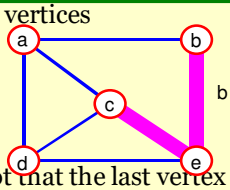
- path: sequence of vertices $v_1, v_2, \ldots v_k$ such that consecutive vertices v_i and v_{i+1} are adjacent.



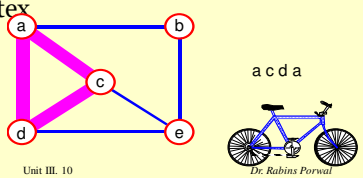
9

More Terminology

- simple path: no repeated vertices

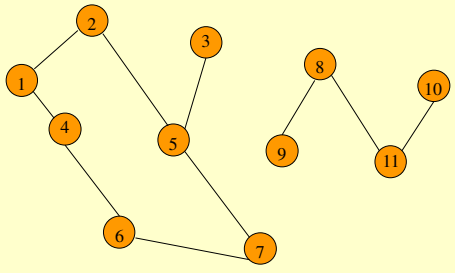


- cycle: simple path, except that the last vertex is the same as the first vertex



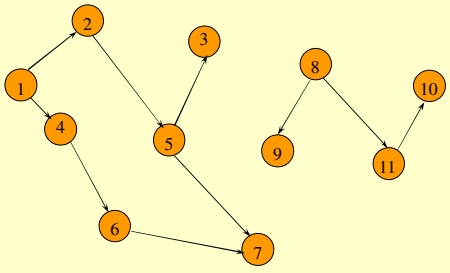
10

Undirected Graph



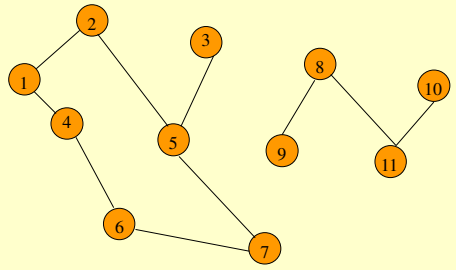
11

Directed Graph (Digraph)



12

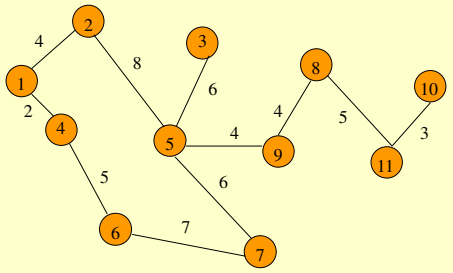
Applications - Communication Network



Vertex = city, *edge* = communication link

13

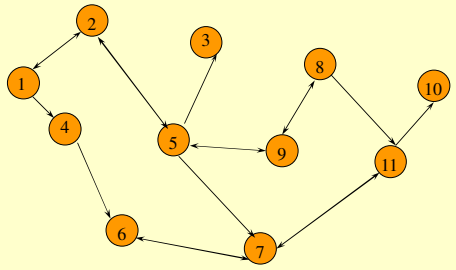
Driving Distance/ Time Map



Vertex = city, edge-weight = driving distance/ time

14

Street Map

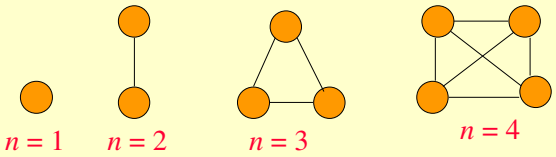


Some streets are one way

15

Complete Undirected Graph

Has all possible edges.



16

Number of Edges - Undirected Graph

- Each edge is of the form (u, v), $u \neq v$.
- Number of such pairs in an n vertex graph is $n(n-1)$.
- Since edge (u, v) is the same as edge (v, u), the number of edges in a complete undirected graph is $n(n-1)/2$.
- Number of edges in an undirected graph is $\leq n(n-1)/2$.

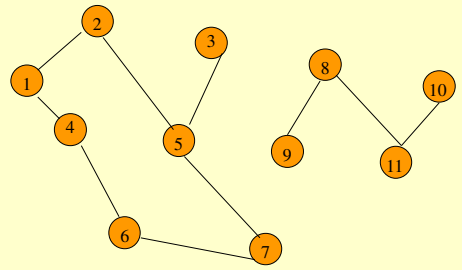
17

Number of Edges - Directed Graph

- Each edge is of the form (u, v), $u \neq v$.
- Number of such pairs in an n vertex graph is $n(n-1)$.
- Since edge (u, v) is not the same as edge (v, u), the number of edges in a complete directed graph is $n(n-1)$.
- Number of edges in a directed graph is $\leq n(n-1)$.

18

Vertex Degree

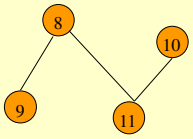


Number of edges incident to vertex.

degree(2) = 2, degree(5) = 3, degree(3) = 1

19

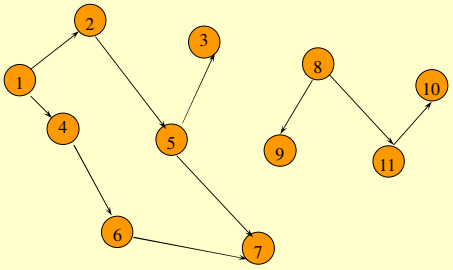
Sum of Vertex Degrees



Sum of degrees = 2×*e* (*e* is number of edges)

20

In-Degree of a Vertex

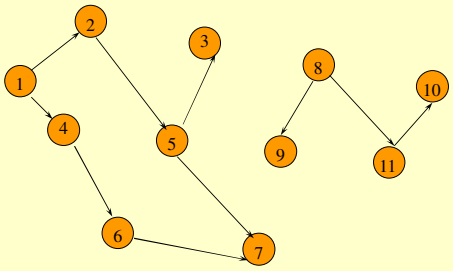


in-degree is number of incoming edges

indegree(2) = 1, indegree(8) = 0

21

Out-Degree of a Vertex



out-degree is number of outgoing edges

outdegree(2) = 1, outdegree(8) = 2

22

Sum of In- and Out-Degrees

Each edge contributes 1 to the in-degree of some vertex and 1 to the out-degree of some other vertex

sum of in-degrees = *sum of out-degrees* = *e*,

where *e* is the number of edges in the digraph

23

Graph Operations And Representation

24

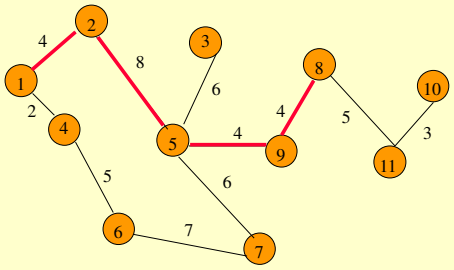
Sample Graph Problems

- Path problems
- Connectedness problems
- Spanning tree problems

25

Path Finding

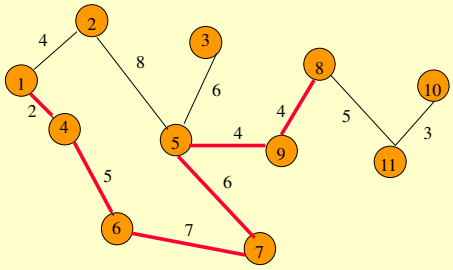
- Path between 1 and 8.



Path length is 20

26

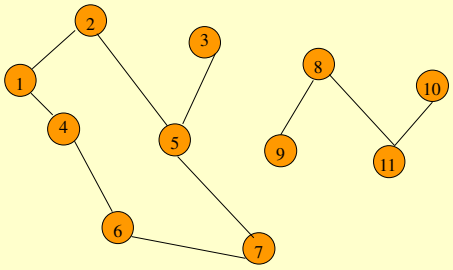
Another Path Between 1 and 8



Path length is 28

27

Example of No Path



No path between 2 and 9

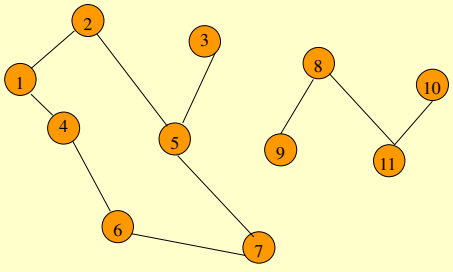
28

Connected Graph

- Undirected graph
- There is a path between every pair of vertices

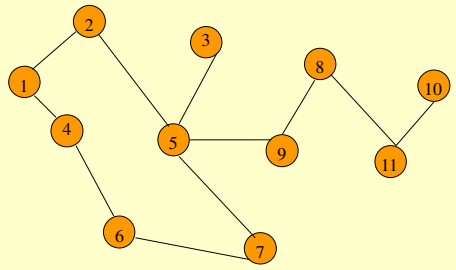
29

Example of Not-Connected



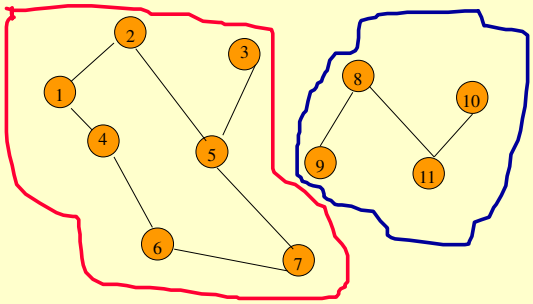
30

Connected Graph Example



31

Connected Components



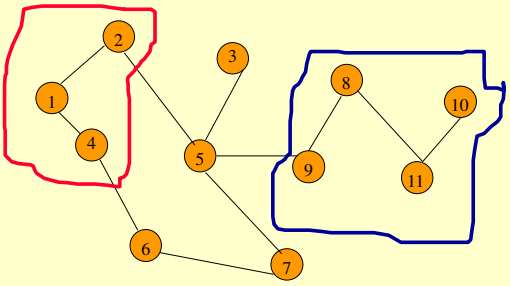
32

Connected Component

- A maximal subgraph that is connected.
  - Cannot add vertices and edges from original graph and retain connectedness.
- A connected graph has exactly 1 component.

33

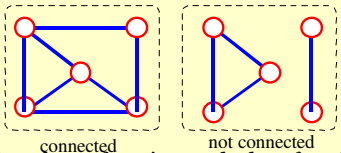
Not A Component



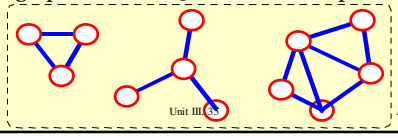
34

Summary

•connected graph: any two vertices are connected by some path

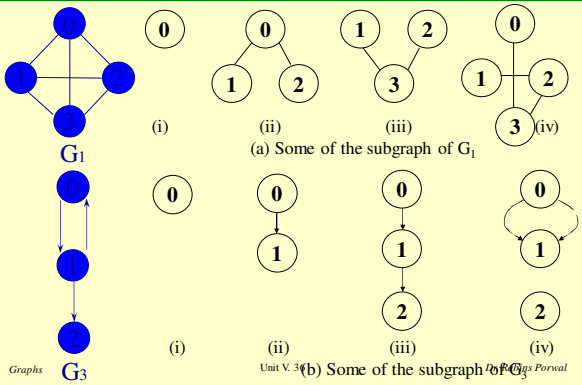


- subgraph: subset of vertices and edges forming a graph
- connected component: maximal connected subgraph, e.g., the graph below has 3 connected components.



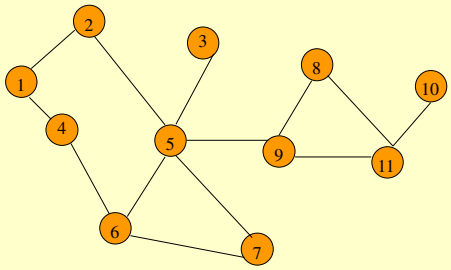
35

Subgraph's Examples



36

Communication Network



Each edge is a link that can be constructed (i.e., a feasible link).

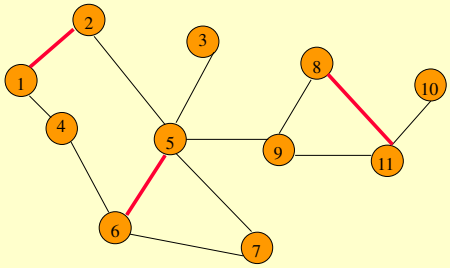
37

Communication Network Problems

- Is the network connected?
  - Can we communicate between every pair of cities?
- Find the components.
- Want to construct smallest number of feasible links so that resulting network is connected.

38

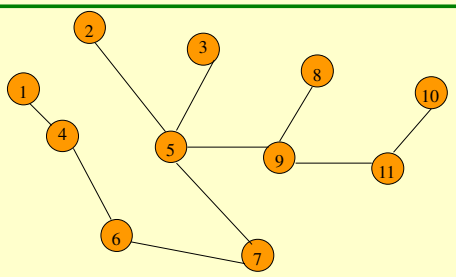
Cycles and Connectedness



Removal of an edge that is on a cycle does not affect connectedness

39

Cycles and Connectedness



Connected subgraph, with all vertices and minimum number of edges, has no cycles.

40

Isomorphic & Cut set

- Two graphs are said to be *isomorphic* if,
  they have the *same number of vertices*
  they have the *same number of edges*
  they have an *equal number of vertices* with a given degree
- A *cut set* in a connected graph G is the set of edges whose removal from G leaves G disconnected, provided the removal of no proper subset of these edges disconnects the graph G.

41

Eulerian walk

- A graph G is called a *labeled graph* if its edges and / or vertices are assigned some data.
- In particular if the edge *e* is assigned a non negative number *l(e)* then it is called the *weight* or *length* of the edge *e*
- A walk starting at any vertex going through each edge exactly once and terminating at the start vertex is called an *Eulerian walk* or *Euler line*

42

Hamiltonian Circuit

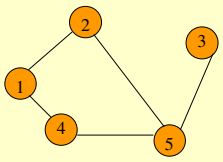
- A Hamiltonian circuit in a connected graph is defined as a closed walk that traverses every vertex of G exactly once, except of course the starting vertex at which the walk terminates
- A circuit in a connected graph G is said to be Hamiltonian if it includes every vertex of G
- If any edge is removed from a Hamiltonian circuit then what remains is referred to as a Hamiltonian path
- Hamiltonian path traverses every vertex of G.

Graph Representation

- Adjacency Matrix
- Adjacency Lists
  - Linked Adjacency Lists
  - Array Adjacency Lists

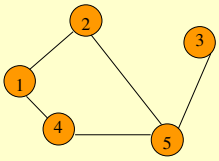
Adjacency Matrix

- 0/1, $n \times n$ matrix, where n = # of vertices
- $A(i, j) = 1$ iff (i, j) is an edge



	1	2	3	4	5
1	0	1	0	1	0
2	1	0	0	0	1
3	0	0	0	0	1
4	1	0	0	0	1
5	0	1	1	1	0

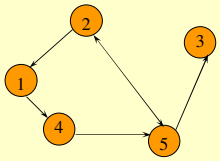
Adjacency Matrix Properties



	1	2	3	4	5
1	0	1	0	1	0
2	1	0	0	0	1
3	0	0	0	0	1
4	1	0	0	0	1
5	0	1	1	1	0

- Diagonal entries are zero.
- Adjacency matrix of an undirected graph is symmetric.
  - $A(i, j) = A(j, i)$ for all i and j

Adjacency Matrix (Digraph)

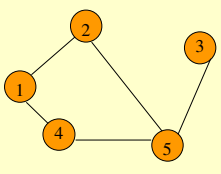


	1	2	3	4	5
1	0	0	0	1	0
2	1	0	0	0	1
3	0	0	0	0	0
4	0	0	0	0	1
5	0	1	1	0	0

- Diagonal entries are zero.
- Adjacency matrix of a digraph need not be symmetric.

Adjacency Lists

- Adjacency list for vertex i is a linear list of vertices adjacent to vertex i.
- An array of n adjacency lists.



aList[1] = (2, 4)

aList[2] = (1, 5)

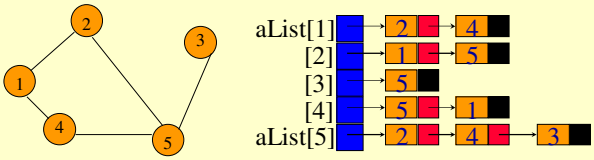
aList[3] = (5)

aList[4] = (5, 1)

aList[5] = (2, 4, 3)

Linked Adjacency Lists

- Each adjacency list is a chain.



Array Length = n

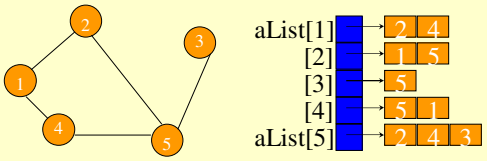
# of chain nodes = $2e$ (undirected graph)

# of chain nodes = e (digraph)

49

Array Adjacency Lists

- Each adjacency list is an array list.



Array Length = n

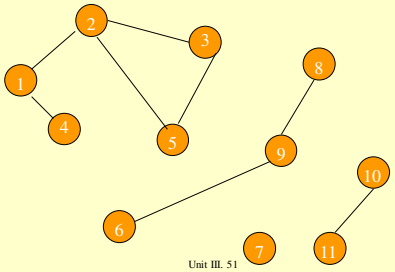
# of list elements = $2e$ (undirected graph)

# of list elements = e (digraph)

50

Graph Search Methods

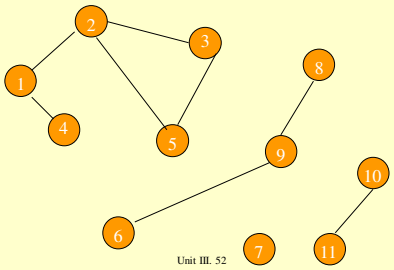
- A vertex u is *reachable* from vertex v iff there is a path from v to u.



51

Graph Search Methods

- A search method starts at a given vertex v and visits/labels/marks every vertex that is reachable from v.



52

Graph Search Methods

- Many graph problems solved using a search method.
  - Path from one vertex to another.
  - Is the graph connected?
  - Find a spanning tree.
  - Etc.
- Commonly used search methods:
  - Depth-first search.
  - Breadth-first search.

53

Depth-First Search

```
dfs(v)
{
  Label vertex v as reached.
  for (each unreached vertex u adjacent to v)
        dfs(u);
}
```

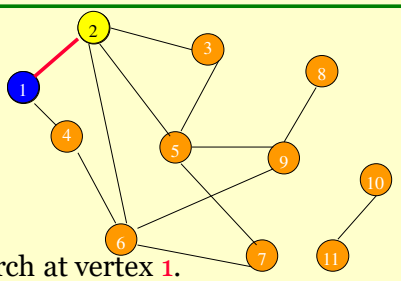

54

DFS Algorithm

- Let G be a graph with starting node A.
  1. Initialize all nodes to the ready state (Status = 1).
  2. Push the starting node A onto Stack and change its status to the waiting state (Status = 2).
  3. Repeat steps 4 and 5 until Stack is empty.
  4. Pop the top node N of the Stack. Process N and change its status to the processed state (Status =3).
  5. Push onto Stack all the neighbors of N that are still in ready state (Status = 1), and change their status to the waiting state (Status = 2).
  6. Exit.

55

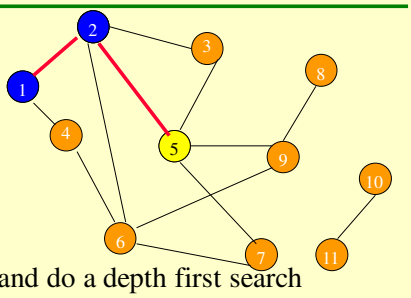
Depth-First Search Example



Start search at vertex 1.
Label vertex 1 and do a depth first search from either 2 or 4.
Suppose that vertex 2 is selected.

56

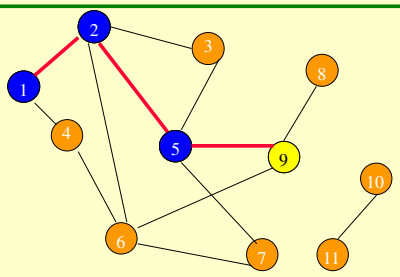
Depth-First Search Example



Label vertex 2 and do a depth first search from either 3, 5, or 6.
Suppose that vertex 5 is selected.

57

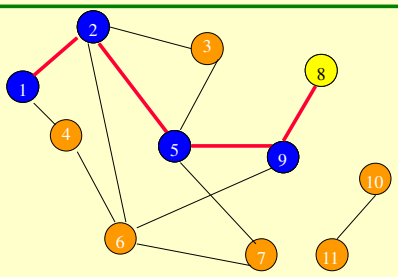
Depth-First Search Example



Label vertex 5 and do a depth first search from either 3, 7, or 9.
Suppose that vertex 9 is selected.

58

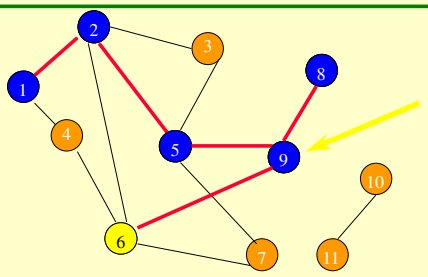
Depth-First Search Example



Label vertex 9 and do a depth first search from either 6 or 8.
Suppose that vertex 8 is selected.

59

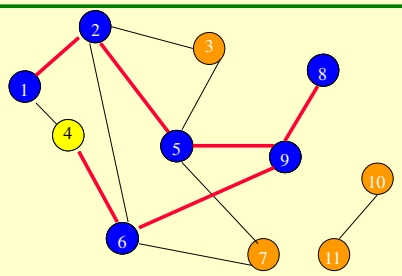
Depth-First Search Example



Label vertex 8 and return to vertex 9.
From vertex 9 do a DFS(6).

60

Depth-First Search Example

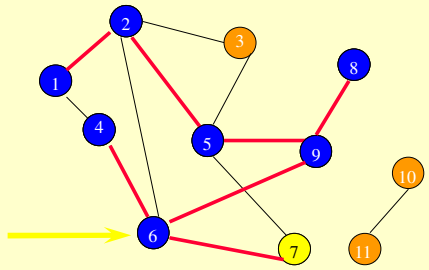


Label vertex 6 and do a depth first search from either 4 or 7.

Suppose that vertex 4 is selected.

61

Depth-First Search Example

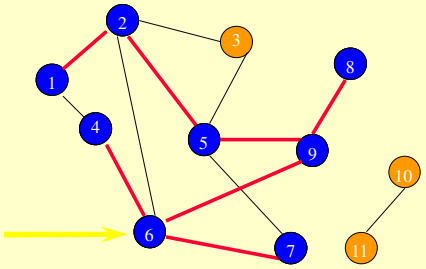


Label vertex 4 and return to 6.

From vertex 6 do a dfs(7).

62

Depth-First Search Example

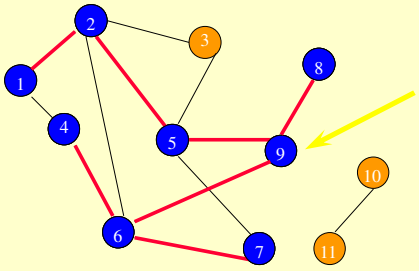


Label vertex 7 and return to 6.

Return to 9.

63

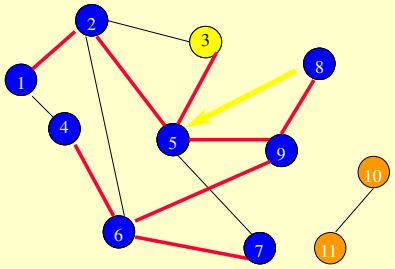
Depth-First Search Example



Return to 5

64

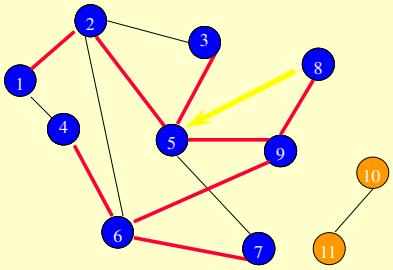
Depth-First Search Example



Do a dfs(3).

65

Depth-First Search Example

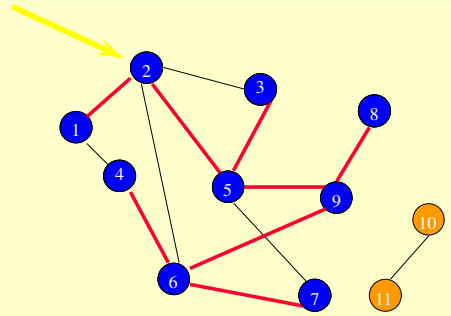


Label 3 and return to 5.

Return to 2.

66

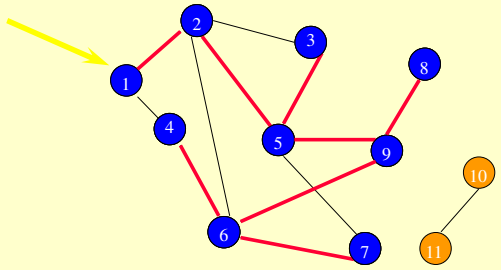
Depth-First Search Example



Return to 1.

67

Depth-First Search Example



Return to invoking method.

68

Depth-First Search Property

- All vertices reachable from the start vertex (including the start vertex) are visited.

69

Path from vertex *v* to vertex *u*

- Start a depth-first search at vertex *v*.
- Terminate when vertex *u* is visited or when dfs ends (whichever occurs first).
- Time
  - $O(n^2)$ when adjacency matrix used
  - $O(n+e)$ when adjacency lists used (e is number of edges)

70

Is the Graph connected?

- Start a depth-first search at any vertex of the graph.
- Graph is connected iff all n vertices get visited.
- Time
  - $O(n^2)$ when adjacency matrix used
  - $O(n+e)$ when adjacency lists used (e is number of edges)

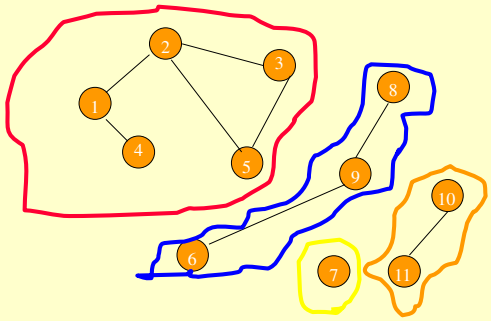
71

Connected Components

- Start a depth-first search at any as yet unvisited vertex of the graph.
- Newly visited vertices (plus edges between them) define a component.
- Repeat until all vertices are visited.

72

Connected Components



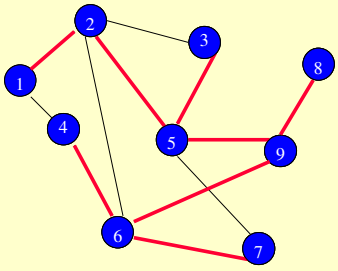
73

Time Complexity

- $O(n^2)$ when adjacency matrix used
- $O(n+e)$ when adjacency lists used (e is number of edges)

74

Spanning Tree



Depth-first search from vertex 1.

Depth-first spanning tree.

75

Spanning Tree

- Start a depth-first search at any vertex of the graph.
- If graph is connected, the n-1 edges used to get to unvisited vertices define a spanning tree (*depth-first spanning tree*).
- Time
  - $O(n^2)$ when adjacency matrix used
  - $O(n+e)$ when adjacency lists used (e is number of edges)

76

Breadth-First Search

- Visit start vertex and put into a FIFO queue.
- Repeatedly remove a vertex from the queue, visit its unvisited adjacent vertices, put newly visited vertices into the queue.

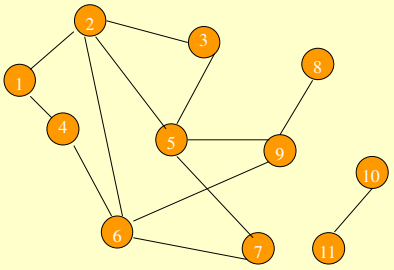
77

BFS Algorithm

- Let G be a graph with starting node A.
  1. Initialize all nodes to the ready state (Status = 1).
  2. Put the starting node A in Queue and change its status to the waiting state (Status = 2).
  3. Repeat steps 4 and 5 until Queue is empty.
  4. Remove the front node N of queue. Process N and change its status to the processed state (Status =3).
  5. Add to the rear of Queue all the neighbors of N that are still in ready state (Status = 1), and change their status to the waiting state (Status = 2).
  6. Exit.

78

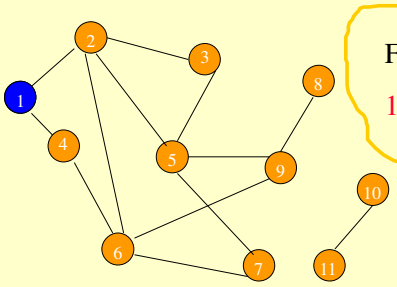
Breadth-First Search Example



Start search at vertex 1.

79

Breadth-First Search Example



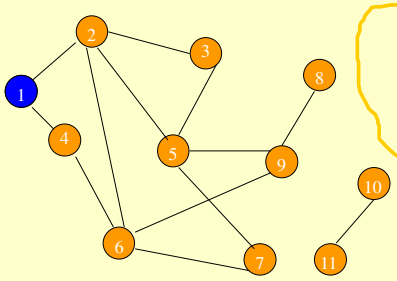
FIFO Queue

1

Visit/mark/label start vertex and put in a FIFO queue.

80

Breadth-First Search Example



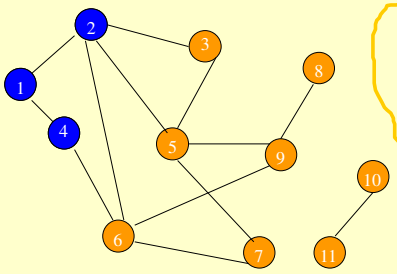
FIFO Queue

1

Remove 1 from Q; visit adjacent unvisited vertices; put in Q.

81

Breadth-First Search Example



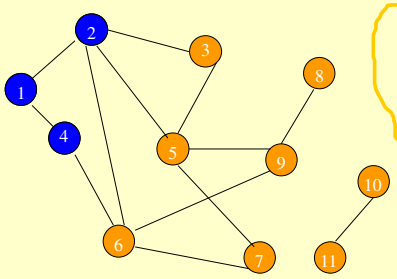
FIFO Queue

2 4

Remove 1 from Q; visit adjacent unvisited vertices; put in Q.

82

Breadth-First Search Example



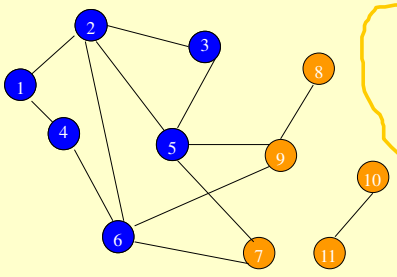
FIFO Queue

2 4

Remove 2 from Q; visit adjacent unvisited vertices; put in Q.

83

Breadth-First Search Example



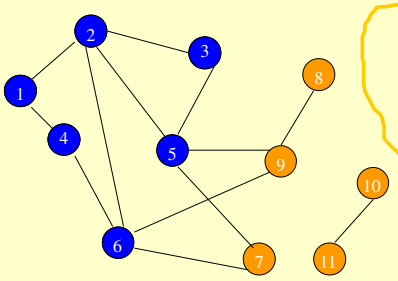
FIFO Queue

4 5 3 6

Remove 2 from Q; visit adjacent unvisited vertices; put in Q.

84

Breadth-First Search Example

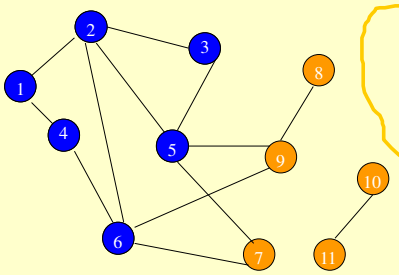


FIFO Queue
4 5 3 6

Remove 4 from Q; visit adjacent unvisited vertices; put in Q.

85

Breadth-First Search Example

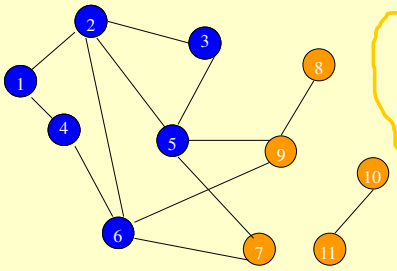


FIFO Queue
5 3 6

Remove 4 from Q; visit adjacent unvisited vertices; put in Q.

86

Breadth-First Search Example

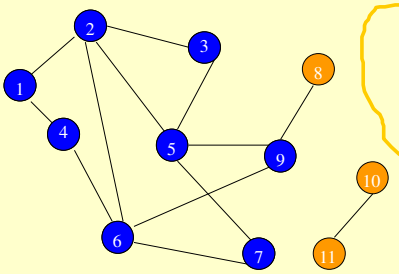


FIFO Queue
5 3 6

Remove 5 from Q; visit adjacent unvisited vertices; put in Q.

87

Breadth-First Search Example

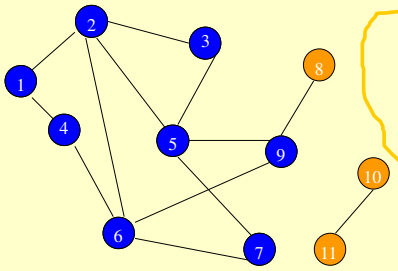


FIFO Queue
3 6 9 7

Remove 5 from Q; visit adjacent unvisited vertices; put in Q.

88

Breadth-First Search Example

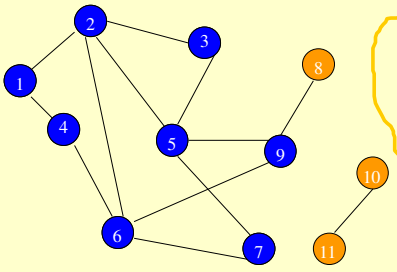


FIFO Queue
3 6 9 7

Remove 3 from Q; visit adjacent unvisited vertices; put in Q.

89

Breadth-First Search Example

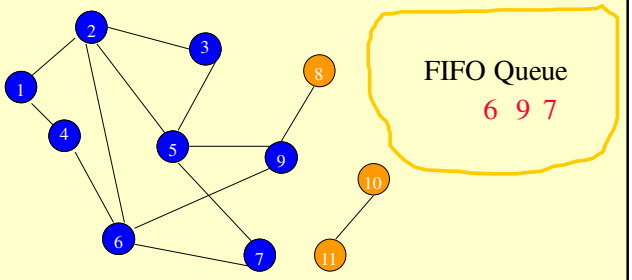


FIFO Queue
6 9 7

Remove 3 from Q; visit adjacent unvisited vertices; put in Q.

90

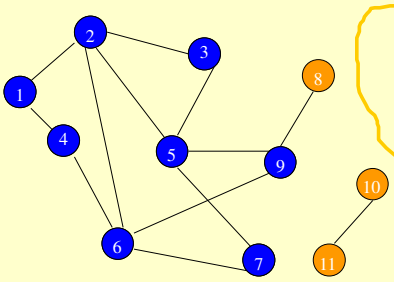
Breadth-First Search Example



Remove 6 from Q; visit adjacent unvisited vertices; put in Q.

91

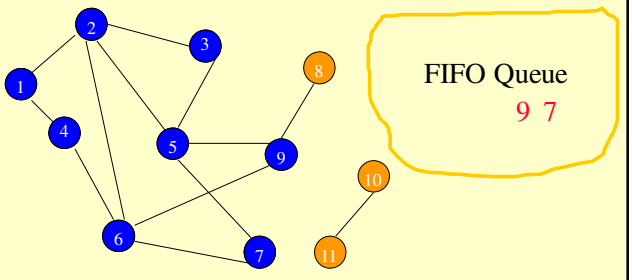
Breadth-First Search Example



Remove 6 from Q; visit adjacent unvisited vertices; put in Q.

92

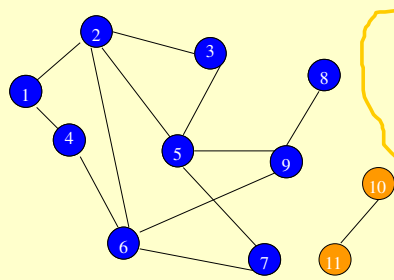
Breadth-First Search Example



Remove 9 from Q; visit adjacent unvisited vertices; put in Q.

93

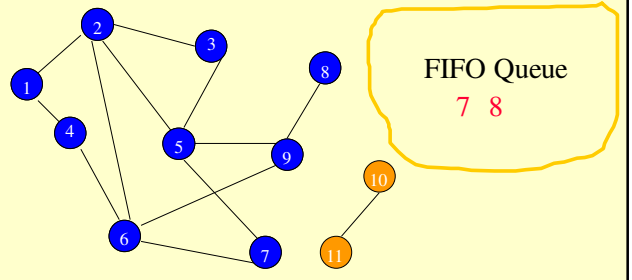
Breadth-First Search Example



Remove 9 from Q; visit adjacent unvisited vertices; put in Q.

94

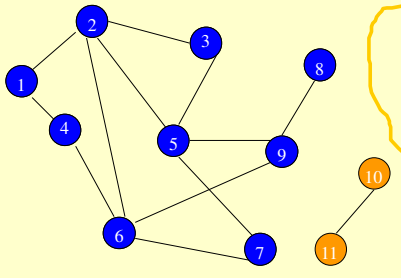
Breadth-First Search Example



Remove 7 from Q; visit adjacent unvisited vertices; put in Q.

95

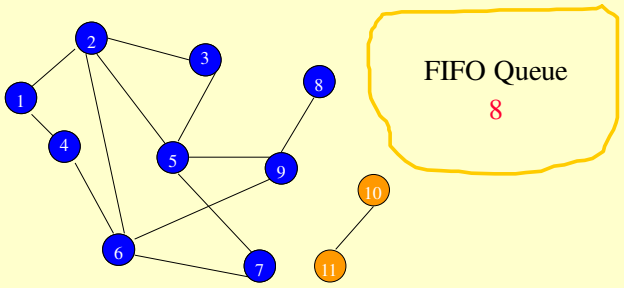
Breadth-First Search Example



Remove 7 from Q; visit adjacent unvisited vertices; put in Q.

96

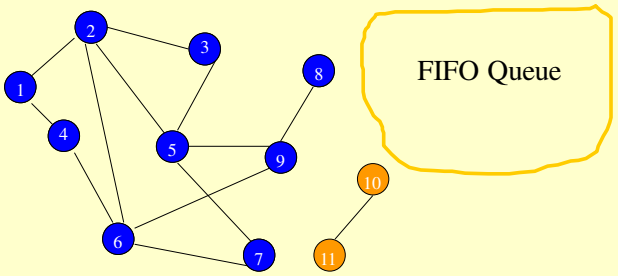
Breadth-First Search Example



Remove 8 from Q; visit adjacent unvisited vertices; put in Q.

97

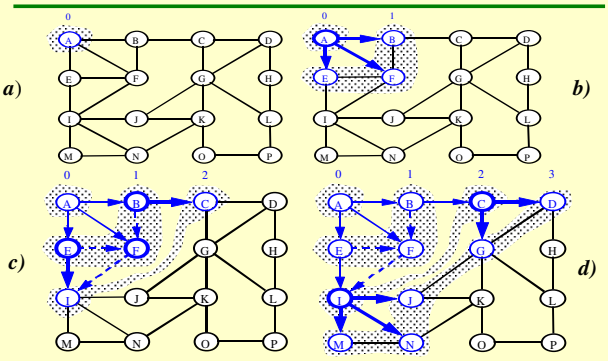
Breadth-First Search Example



Queue is empty. Search terminates.

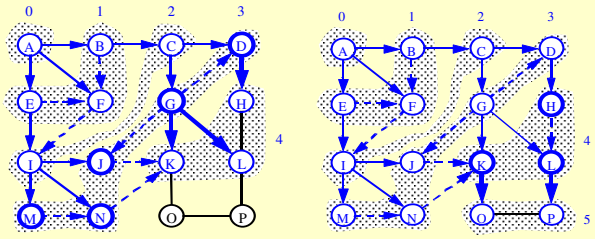
98

BFS - A Graphical Representation



99

More BFS



100

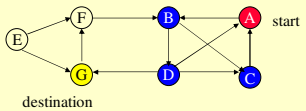
Applications: Finding a Path

- Find path from source vertex *s* to destination vertex *d*
- Use graph search starting at *s* and terminating as soon as we reach *d*
  - Need to remember edges traversed
- Use depth first search ?
- Use breath first search?

101

DFS vs. BFS

DFS Process



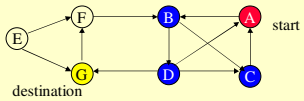
A	DFS on A	B A	DFS on B	C B A	DFS on C	D B A	Call DFS on D Return to call on B

G D B A	Call DFS on G	found destination - done! Path is implicitly stored in DFS recursion Path is: A, B, D, G

102

DFS vs. BFS

BFS Process



rear front	rear front	rear front	rear front
A	B	C D	D
Initial call to BFS on A Add A to queue	Dequeue A Add B	Dequeue B Add C, D	Dequeue C Nothing to add

rear front
G
Dequeue D Add G

found destination - done!
Path must be stored separately

103

Time Complexity

- Each visited vertex is put on (and so removed from) the queue exactly once.
- When a vertex is removed from the queue, we examine its adjacent vertices.
  - $O(n)$ if adjacency matrix used
  - $O(\textit{vertex degree})$ if adjacency lists used
- Total time
  - $O(m \times n)$, where m is number of vertices in the component that is searched (adjacency matrix)

104

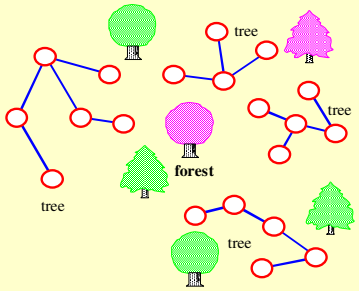
Breadth-First Search Properties

- Same complexity as *dfs*.
- Same properties with respect to path finding, connected components, and spanning trees.
- Edges used to reach unlabeled vertices define a *Breadth-first spanning tree* when the graph is connected.
- There are problems for which *bfs* is better than *dfs* and vice versa.

105

Trees & Forests

- Tree - connected graph without cycles
- Forest - collection of trees



106

Tree

- Connected graph that has no cycles.
- n vertex connected graph with n-1 edges.

107

Spanning Tree

- Subgraph that includes all vertices of the original graph.
- Subgraph is a tree.
  - If original graph has n vertices, the spanning tree has n vertices and n-1 edges.

108

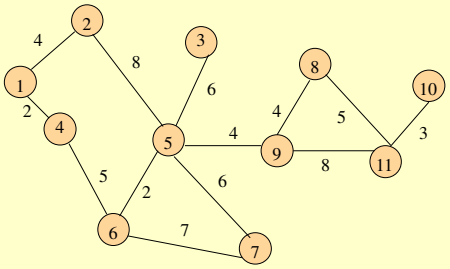
Spanning Tree

- Let G be a weighted connected graph. A spanning tree of the graph G contains all the nodes and has the edges, which connect all the nodes. So number of edges will be 1 less than the number of nodes.
- If a graph is not connected, *i.e.*, a graph with n nodes having edges less than $n - 1$, then no spanning tree is possible.
- DFS and BFS spanning trees are spanning trees which are obtained by DFS and BFS traversals.

Spanning Tree (Imp. Points)

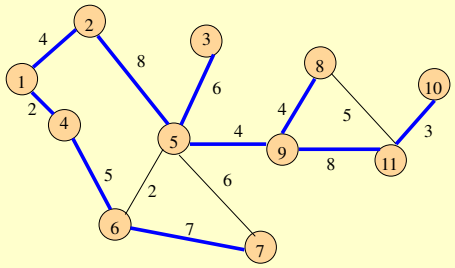
- Trees can be defined as special cases of graphs. A tree may be defined as a connected graph without any cycle.
- The difference between general trees and trees defined here is that a tree defined as a special case of graph does not have special vertex called *root*. In fact any vertex can be chosen as root.

Cost of Spanning Tree



- Tree cost is sum of edge weights/costs

A Spanning Tree

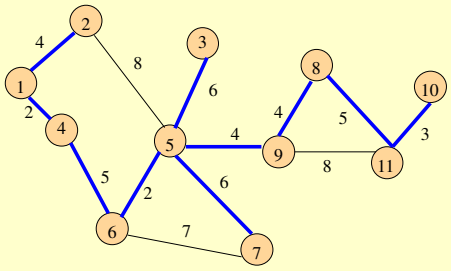


Spanning tree cost = 51

Minimum-Cost Spanning Tree

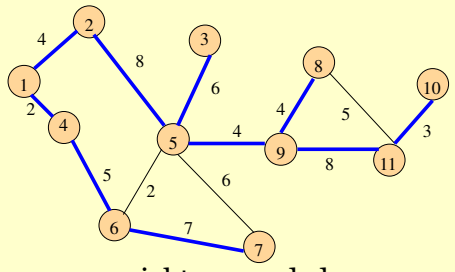
- weighted connected undirected graph
- spanning tree
- cost of spanning tree is sum of edge costs
- A spanning tree that has *minimum cost*

Minimum Cost Spanning Tree



Spanning tree cost = 41

A Wireless Broadcast Tree

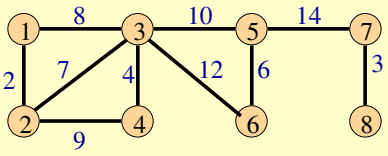


Source = 1, weights = needed power

Cost = 4 + 8 + 5 + 6 + 7 + 8 + 3 = 41

115

Another Example



- Network has 10 edges.
- Spanning tree has only $n - 1 = 7$ edges.
- Need to either select 7 edges or discard 3.

116

Edge Selection Strategies

- Start with an n-vertex o-edge forest. Consider edges in ascending order of cost. Select edge if it does not form a cycle together with already selected edges.
  - Kruskal's method.
- Start with a 1-vertex tree and grow it into an n-vertex tree by repeatedly adding a vertex and an edge. When there is a choice, add a least cost edge.
  - Prim's method.

117

Edge Rejection Strategies

- Start with the connected graph. Repeatedly find a cycle and eliminate the highest cost edge on this cycle. Stop when no cycles remain.
- Consider edges in descending order of cost. Eliminate an edge provided this leaves behind a connected graph.

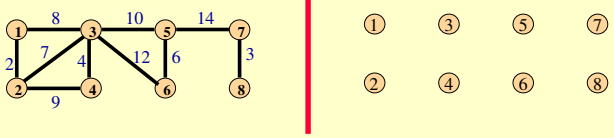
118

Kruskal's Algorithm

- This algorithm works on a list of edges of a graph where the list is arranged in ascending order of weight of the edges.
  1. Make a forest of 'n' trees if the graph has n vertices. The trees in the forest contain a different vertex of the graph and no edges.
  2. Create a sorted list of edges.
  3. Take one edge of minimum weight from the sorted list of edges for connecting two trees in the forest. The edge is added if this incorporation does not form a cycle. Once the edge is selected, it is deleted from the list of edges.
  4. The algorithm continues until $(n - 1)$ edges are added or the list of edges exhausted.
  5. After adding $(n - 1)$ edges, the algorithm ends and a minimum spanning tree is produced.

119

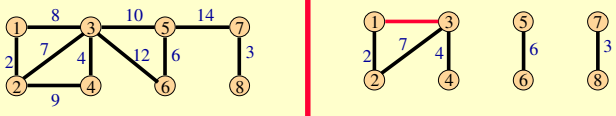
Kruskal's Method



- Start with a forest that has no edges.
- Consider edges in ascending order of cost.
- Edge (1, 2) is considered first and added to the forest.

120

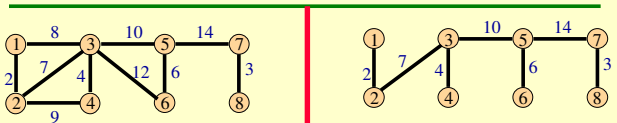
Kruskal's Method



- Edge (7, 8) is considered next and added.
- Edge (3, 4) is considered next and added.
- Edge (5, 6) is considered next and added.
- Edge (2, 3) is considered next and added.
- Edge (1, 3) is considered next and rejected because it creates a cycle.

121

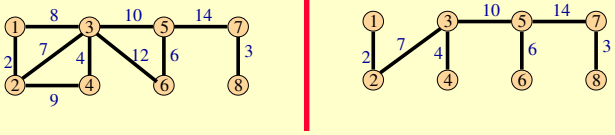
Kruskal's Method



- Edge (2, 4) is considered next and rejected because it creates a cycle.
- Edge (3, 5) is considered next and added.
- Edge (3, 6) is considered next and rejected.
- Edge (5, 7) is considered next and added.

122

Kruskal's Method



- n - 1 edges have been selected and no cycle formed.
- So we must have a spanning tree.
- Cost is 46.
- Min-cost spanning tree is unique when all edge costs are different.

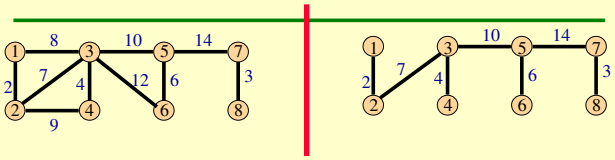
123

Prim's Algorithm

1. Start with any arbitrary vertex as the partial minimum spanning tree T.
2. Add one edge (u, v) of the minimum weight/ least cost to the partial tree T so that exactly one end of this edge belongs to the set of vertices in T.
3. The process is repeated until $(n - 1)$ edges are added.

124

Prim's Method



- Start with any single vertex tree.
- Get a 2-vertex tree by adding a cheapest edge.
- Get a 3-vertex tree by adding a cheapest edge.
- Grow the tree one edge at a time until the tree has n - 1 edges (and hence has all n vertices).

125

Minimum-Cost Spanning Tree Methods

- Can prove that all stated edge selection/ rejection result in a minimum-cost spanning tree.
- Prim's method is fastest.
  - $O(n^2)$ using an implementation similar to that of Dijkstra's shortest-path algorithm.
  - $O(e + n \log n)$ using a Fibonacci heap.
- Kruskal's uses union-find trees to run in $O(n + e \log e)$ time.

126

Pseudocode For Kruskal's Method

- Start with an empty set T of edges.
- while (E is not empty && $|T| \neq n-1$)
- {
- Let (u, v) be a least-cost edge in E.
- $E = E - \{(u, v)\}$. // delete edge from E
- if (u, v) does not create a cycle in T)
- Add edge (u, v) to T.
- }
- if ($| T | == n-1$) T is a min-cost spanning tree.
- else Network has no spanning tree.

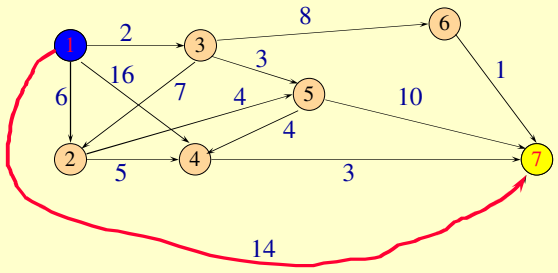
127

Shortest Path Problems

- Directed weighted graph.
- Path length is sum of weights of edges on path.
- The vertex at which the path begins is the source vertex.
- The vertex at which the path ends is the destination vertex.

128

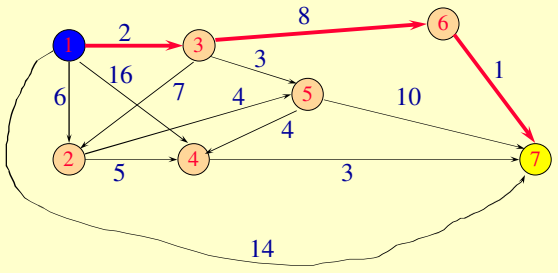
Example



A path from 1 to 7.
Path length is 14.

129

Example



Another path from 1 to 7.
Path length is 11.

130

Shortest Path Problems

- Single source single destination.
- Single source all destinations.
- All pairs (every vertex is a source and destination).

131

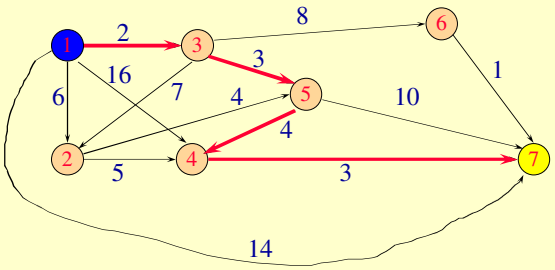
Single Source Single Destination

Possible algorithm:

- Leave source vertex using cheapest/shortest edge.
- Leave new vertex using cheapest edge subject to the constraint that a new vertex is reached.
- Continue until destination is reached.

132

Constructed 1 to 7 Path



Path length is 12.

Not shortest path. Algorithm doesn't work!

133

Single Source All Destinations

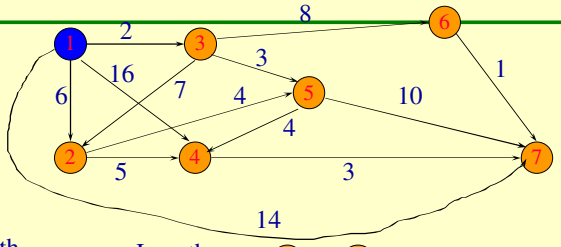
Need to generate up to n (n is number of vertices) paths (including path from source to itself).

Dijkstra's method:

- Construct these up to n paths in order of increasing length.
- Assume edge costs (lengths) are ≥ 0.
- So, no path has length < 0.
- First shortest path is from the source vertex to itself. The length of this path is 0.

134

Single Source All Destinations



Path	Length
1	0
1 3	2
1 3 5	5
1 2	6
1 3 5 4	9
1 3 6	10
1 3 6 7	11

135

Single Source All Destinations

Path	Length
1	0
1 3	2
1 3 5	5
1 2	6
1 3 5 4	9
1 3 6	10
1 3 6 7	11

- Each path (other than first) is a one edge extension of a previous path.
- Next shortest path is the shortest one edge extension of an already generated shortest path.

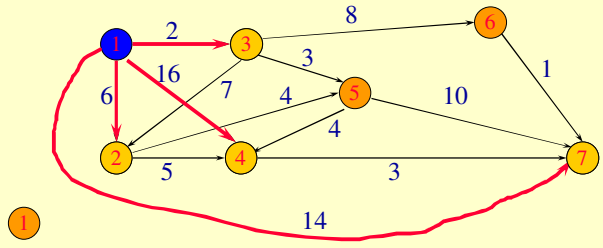
136

Single Source All Destinations

- Let $d[i]$ be the length of a shortest one edge extension of an already generated shortest path, the one edge extension ends at vertex i.
- The next shortest path is to an as yet unreached vertex for which the $d[\ ]$ value is least.
- Let $p[i]$ be the vertex just before vertex i on the shortest one edge extension to i.

137

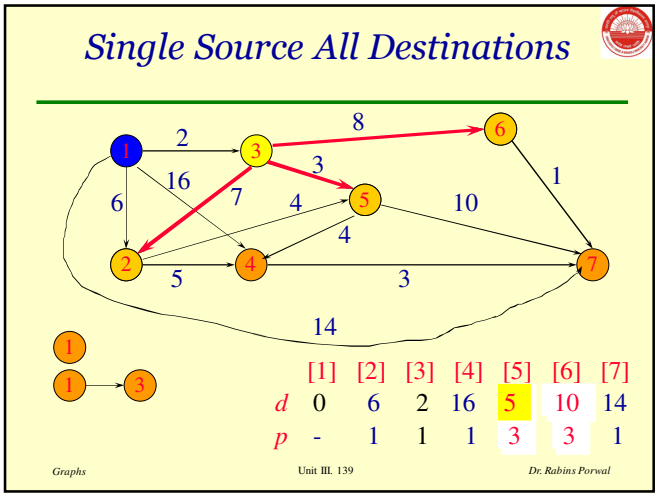
Single Source All Destinations



	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	16	-	-	14
p	-	1	1	1	-	-	1

138

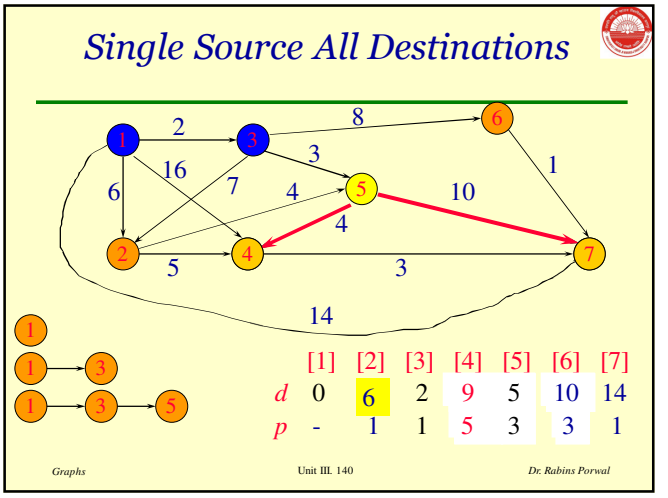
Single Source All Destinations



	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	16	5	10	14
p	-	1	1	1	3	3	1

139

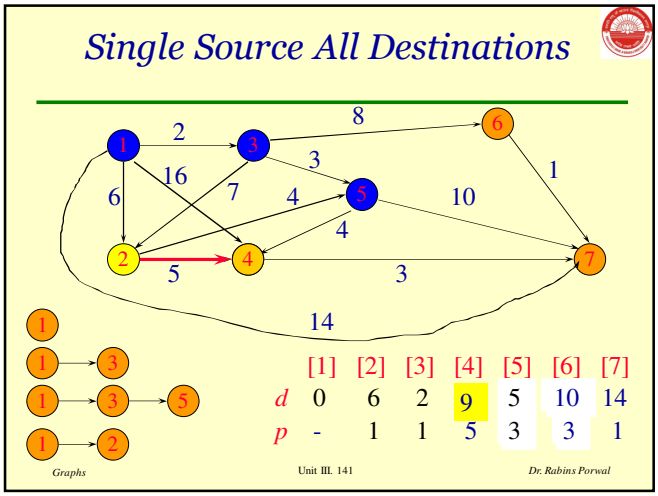
Single Source All Destinations



	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	9	5	10	14
p	-	1	1	5	3	3	1

140

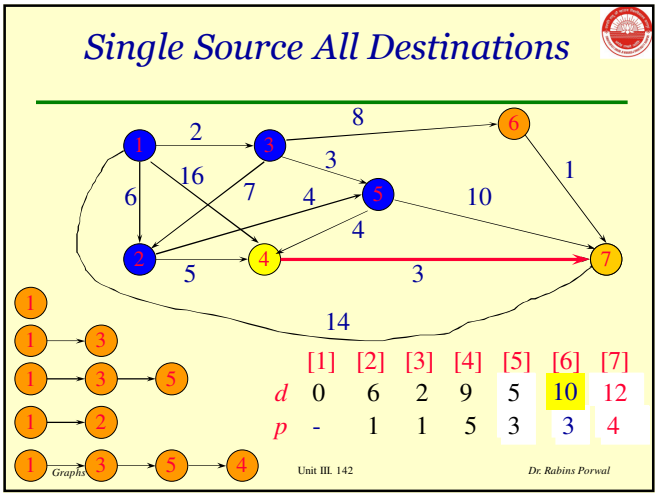
Single Source All Destinations



	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	9	5	10	14
p	-	1	1	5	3	3	1

141

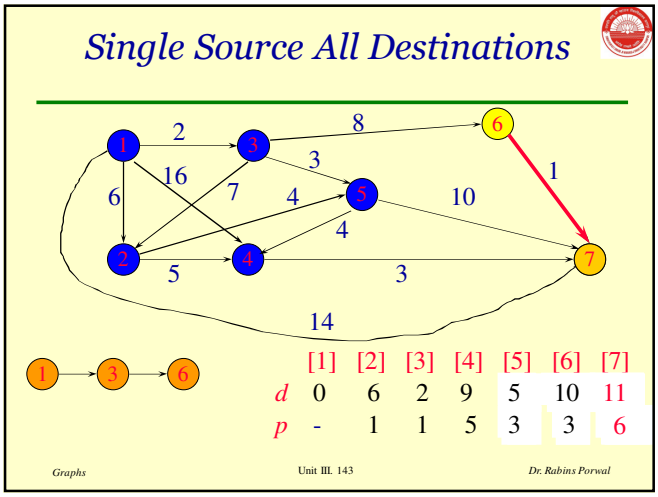
Single Source All Destinations



	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	9	5	10	12
p	-	1	1	5	3	3	4

142

Single Source All Destinations



	[1]	[2]	[3]	[4]	[5]	[6]	[7]
d	0	6	2	9	5	10	11
p	-	1	1	5	3	3	6

143

Single Source All Destinations

Path	Length
1	0
1 → 3	2
1 → 3 → 5	5
1 → 2	6
1 → 3 → 5 → 4	9
1 → 3 → 6	10
1 → 3 → 6 → 7	11

	[1]	[2]	[3]	[4]	[5]	[6]	[7]
	0	6	2	9	5	10	11
	-	1	1	5	3	3	6

144

Single Source All Destination

Terminate single source all destinations algorithm as soon as shortest path to desired vertex has been generated.

145

Data Structures For Dijkstra's Algorithm

- The described single source all destinations algorithm is known as Dijkstra's algorithm.
- Implement $d[\ ]$ and $p[\ ]$ as 1D arrays.
- Keep a linear list L of reachable vertices to which shortest path is yet to be generated.
- Select and remove vertex v in L that has smallest $d[\ ]$ value.
- Update $d[\ ]$ and $p[\ ]$ values of vertices adjacent to v.

146

Thank you

Questions???

147